

Optimización y Análisis de Redes

Apuntes y
Laboratorios

Grado en Matemáticas

AUTORES

- Víctor Aceña Gil
- Emilio López Cano
- Javier Martínez Moguerza

2026-2027



Índice de Contenidos

Introducción	5
Estructura de los Laboratorios	5
Requisitos y Preparación del Entorno	5
1 Introducción a la Optimización	7
1.1 Formulación Matemática General	7
1.1.1 El Conjunto Factible	7
1.2 Clasificación de Problemas de Optimización	8
1.3 Óptimos Locales y Globales	8
2 Optimización Sin Restricciones	10
2.1 Fundamentos Matemáticos: Gradiente y Hessiana	10
2.1.1 Aproximación de Taylor	11
2.2 Condiciones de Optimalidad	11
2.3 Algoritmos de Resolución	11
2.3.1 1. Descenso de Gradiente (Gradient Descent)	11
2.3.2 2. Método de Newton	12
2.4 Optimización Científica con SciPy	13
3 Análisis Convexo	14
3.1 Conjuntos Convexos	14
3.1.1 Ejemplos Clásicos de Conjuntos Convexos	14
3.2 Funciones Convexas	15
3.2.1 Caracterización mediante Derivadas	15
3.3 Operaciones que Preservan la Convexidad	16
3.3.1 En Conjuntos:	16
3.3.2 En Funciones:	16
4 Optimización con Restricciones y Condiciones KKT	17
4.1 Formulación del Problema	17
4.1.1 Restricciones Activas e Inactivas	17
4.2 La Función Lagrangiana y Multiplicadores de Lagrange	18
4.3 Condiciones de Karush-Kuhn-Tucker (KKT)	18
4.3.1 Teorema de KKT (Condiciones Necesarias)	18
4.3.2 Condiciones Suficientes de KKT bajo Convexidad	19

4.4	Dualidad Lagrangiana	19
4.4.1	La Función Dual de Lagrange	19
4.4.2	El Problema Dual de Lagrange	19
5	Programación Lineal, Cuadrática y CVXPY	20
5.1	Programación Lineal (LP)	20
5.1.1	Condiciones KKT para LP	20
5.2	Programación Cuadrática (QP)	21
5.2.1	Condiciones KKT para QP	21
5.3	Resolución de Problemas en Python con CVXPY	21
5.3.1	Ejemplo 1: Resolución de una Programación Lineal (LP)	21
5.3.2	Ejemplo 2: Resolución de una Programación Cuadrática (QP)	22
6	Optimización en Ciencia de Datos: Regresión y SVM	24
6.1	Regresión Lineal y Regularización	24
6.2	Máquinas de Vectores de Soporte (SVM)	24
6.2.1	1. Caso Linealmente Separable (Margen Máximo)	25
6.2.2	2. Caso No Separable (Margen Blando)	25
6.2.3	3. Formulación Dual de SVM	26
7	Introducción al Análisis de Redes y NetworkX	27
7.1	Conceptos Básicos de Teoría de Grafos	27
7.1.1	Tipos de Grafos	27
7.2	Representación de Grafos en Ordenador	27
7.3	Introducción a NetworkX en Python	28
8	Problemas de Camino Mínimo	30
8.1	Algoritmos Fundamentales	30
8.1.1	1. Algoritmo de Dijkstra	30
8.1.2	2. Algoritmo de Bellman-Ford	30
8.1.3	3. Algoritmo de Floyd-Warshall	30
8.2	Resolución de Caminos Mínimos con NetworkX	31
9	Problemas de Flujo en Redes	33
9.1	El Problema de Flujo Máximo	33
9.1.1	Algoritmo de Ford-Fulkerson	33
9.1.2	Teorema de Flujo Máximo - Corte Mínimo	33
9.2	Problema de Flujo a Coste Mínimo (Min-Cost Flow)	34
9.3	Modelado y Resolución con NetworkX	34
10	Problemas de Emparejamiento (Matching)	35
10.1	Tipos de Emparejamientos	35
10.2	Emparejamiento en Grafos Bipartitos	35
10.3	Resolución de Emparejamientos con NetworkX	36

11 Problemas de Recorridos (Tours)	37
11.1 Recorridos Eulerianos y el Cartero Chino	37
11.2 Recorridos Hamiltonianos y el Viajante de Comercio (TSP)	37
11.3 Implementación en Python con NetworkX	38

Introducción

Este libro digital recopila los laboratorios prácticos de la asignatura **Optimización y Análisis de Redes** para el Grado en Matemáticas de la Universidad Rey Juan Carlos.

El objetivo principal de esta colección es guiar al estudiante en la formulación, análisis y resolución computacional de problemas de optimización matemática y redes utilizando el lenguaje de programación **Python**.

Estructura de los Laboratorios

Los temas están divididos en dos grandes bloques:

1. **Optimización No Lineal (Temas 1 a 6):** Desde la modelización inicial y la optimización numérica sin restricciones (como el descenso de gradiente), pasando por la teoría del análisis convexo y las condiciones de optimalidad de Karush-Kuhn-Tucker (KKT), hasta la resolución práctica restringida mediante la librería **CVXPY** y su aplicación directa en problemas de Ciencia de Datos (regresión y Máquinas de Vectores de Soporte).
2. **Optimización en Redes (Temas 7 a 11):** Introducción a la representación de grafos y uso de **NetworkX**, resolución de caminos mínimos (Dijkstra, Bellman-Ford, Floyd-Warshall), problemas de flujo óptimo y corte mínimo, emparejamientos máximos sobre grafos bipartitos, y enrutamiento óptimo (recorridos eulerianos y hamiltonianos, incluyendo el problema del viajante).

Requisitos y Preparación del Entorno

Para ejecutar los ejemplos de código y realizar los ejercicios propuestos en cada tema, deberás configurar tu entorno de trabajo. Se recomienda utilizar el entorno de Conda `oar_env` provisto en el curso.

Las principales librerías utilizadas son:

- **NumPy:** Para cálculo vectorial y matricial.
- **Matplotlib:** Para la generación de gráficos en 2D y 3D.
- **CVXPY:** Para formular y resolver problemas de optimización convexa.
- **NetworkX:** Para crear, manipular y analizar la estructura de redes complejas.

- **Scikit-learn:** Para realizar comparativas de modelos de aprendizaje automático (SVM).

1 Introducción a la Optimización

La **optimización matemática** (o programación matemática) es la disciplina que se ocupa de seleccionar, de acuerdo con algún criterio, el mejor elemento de un conjunto de alternativas disponibles. En términos prácticos, consiste en maximizar o minimizar una función real eligiendo sistemáticamente valores de variables de decisión dentro de un conjunto permitido.

1.1 Formulación Matemática General

Un problema de optimización continua se expresa matemáticamente de la siguiente forma general:

$$\begin{array}{ll} \underset{x}{\text{minimizar}} & f(x) \\ \text{sujeto a} & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{array}$$

Donde: - $x \in \mathbb{R}^n$ es el **vector de variables de decisión** ($x = (x_1, x_2, \dots, x_n)^T$). - $f : \mathbb{R}^n \rightarrow \mathbb{R}$ es la **función objetivo** que se desea minimizar (o maximizar, ya que maximizar $f(x)$ es equivalente a minimizar $-f(x)$). - $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$ son las **restricciones de desigualdad**. - $h_j : \mathbb{R}^n \rightarrow \mathbb{R}$ son las **restricciones de igualdad**.

1.1.1 El Conjunto Factible

El conjunto de todos los puntos que satisfacen todas las restricciones se denomina **conjunto factible** (o región factible), denotado por Ω :

$$\Omega = \{x \in \mathbb{R}^n \mid g_i(x) \leq 0, \quad i = 1, \dots, m; \quad h_j(x) = 0, \quad j = 1, \dots, p\}$$

Si $\Omega = \emptyset$, el problema se dice que es **incompatible** o **infactible**. Si no hay restricciones ($m = 0, p = 0$), el problema se denomina **no restringido** y $\Omega = \mathbb{R}^n$.

1.2 Clasificación de Problemas de Optimización

Los problemas de optimización se clasifican según las propiedades matemáticas de sus funciones y variables:

1. **Optimización Lineal (LP):** La función objetivo $f(x)$ y todas las restricciones $g_i(x)$ y $h_j(x)$ son combinaciones lineales de las variables.
2. **Optimización No Lineal (NLP):** La función objetivo o alguna de las restricciones no es lineal.
3. **Optimización Convexa:** El conjunto factible Ω es un conjunto convexo y la función objetivo $f(x)$ es una función convexa. (Veremos en detalle esta clase de problemas en el Capítulo 3, ya que son de vital importancia debido a que cualquier óptimo local es global).
4. **Optimización Continua vs. Discreta:** En la optimización continua, las variables pueden tomar cualquier valor real dentro de un intervalo. En la discreta (o entera), algunas o todas las variables deben tomar valores enteros (como binarios 0 o 1).

1.3 Óptimos Locales y Globales

i Definición de Mínimo Global

Un punto $x^* \in \Omega$ es un **mínimo global** del problema si:

$$f(x^*) \leq f(x) \quad \forall x \in \Omega$$

i Definición de Mínimo Local

Un punto $x^* \in \Omega$ es un **mínimo local** si existe un entorno de radio $\epsilon > 0$ alrededor de x^* (denotado por $B(x^*, \epsilon)$) tal que:

$$f(x^*) \leq f(x) \quad \forall x \in \Omega \cap B(x^*, \epsilon)$$

En general, encontrar un mínimo global para problemas no lineales no convexos es extremadamente difícil, ya que los algoritmos numéricos basados en derivadas locales suelen quedar atrapados en mínimos locales.

```
# Ejemplo de visualización de mínimos locales vs globales en Python
import numpy as np
import matplotlib.pyplot as plt
```



```

# Definimos una función no convexa con múltiples óptimos
def f(x):
    return x**2 + 10 * np.sin(x)

x = np.linspace(-10, 10, 1000)
y = f(x)

plt.figure(figsize=(8, 5))
plt.plot(x, y, label='$f(x) = x^2 + 10 \sin(x)$', color='darkblue', linewidth=2)

# Destacar mínimo local y global
plt.scatter(-1.3, f(-1.3), color='orange', s=100, zorder=5, label='Mínimo Global')
plt.scatter(4.9, f(4.9), color='red', s=100, zorder=5, label='Mínimo Local')

plt.title('Óptimos Locales y Globales')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

```

2 Optimización Sin Restricciones

La **optimización sin restricciones** aborda la búsqueda de mínimos o máximos de una función real $f(x)$ definida en todo el espacio $\mathbb{R}^n$, es decir, sin condiciones sobre los valores de las variables.

$$\underset{x \in \mathbb{R}^n}{\text{minimizar}} \quad f(x)$$

2.1 Fundamentos Matemáticos: Gradiente y Hessiana

Para funciones multivariantes diferenciables $f : \mathbb{R}^n \rightarrow \mathbb{R}$, las derivadas de primer y segundo orden se organizan en el **gradiente** y la **matriz Hessiana**:

1. **Gradiente** ($\nabla f(x)$): Es el vector columna de las derivadas parciales de primer orden:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^T$$

El gradiente apunta en la dirección de máximo crecimiento local de la función.

2. **Matriz Hessiana** ($\nabla^2 f(x)$): Es la matriz cuadrada de las segundas derivadas parciales:

$$\nabla^2 f(x) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$

Bajo continuidad (Teorema de Clairaut), la matriz Hessiana es simétrica.

2.1.1 Aproximación de Taylor

La aproximación cuadrática de $f(x)$ alrededor de un punto x_k viene dada por:

$$f(x_k + p) \approx f(x_k) + \nabla f(x_k)^T p + \frac{1}{2} p^T \nabla^2 f(x_k) p$$

donde $p \in \mathbb{R}^n$ es el vector de perturbación o dirección de paso.

2.2 Condiciones de Optimalidad

Para que un punto x^* sea un mínimo local de $f(x)$:

- **Condición Necesaria de Primer Orden (FONC):** El punto debe ser estacionario (gradiente nulo):

$$\nabla f(x^*) = 0$$

- **Condición Necesaria de Segundo Orden (SONC):** La matriz Hessiana debe ser semidefinida positiva:

$$\nabla^2 f(x^*) \succeq 0$$

- **Condición Suficiente de Segundo Orden (SOSC):** Si $\nabla f(x^*) = 0$ y la matriz Hessiana es **definida positiva** ($\nabla^2 f(x^*) \succ 0$), entonces x^* es un mínimo local estricto.
-

2.3 Algoritmos de Resolución

2.3.1 1. Descenso de Gradiente (Gradient Descent)

El método del descenso de gradiente es un algoritmo iterativo que se mueve en la dirección opuesta al gradiente (máximo descenso):

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

Donde $\alpha_k > 0$ es la **tasa de aprendizaje** (o longitud de paso).

```
# Implementación paso a paso de Descenso de Gradiente en Python
import numpy as np

def descenso_gradiente(f, grad_f, x_init, tasa_aprendizaje=0.1, max_iter=100, tol=1e-5):
    x = np.array(x_init, dtype=float)
    historial = [x.copy()]

    for i in range(max_iter):
        grad = grad_f(x)
        if np.linalg.norm(grad) < tol:
            print(f"Convergencia alcanzada en la iteración {i}")
            break
        x = x - tasa_aprendizaje * grad
        historial.append(x.copy())

    return x, np.array(historial)
```

2.3.2 2. Método de Newton

El método de Newton utiliza la aproximación cuadrática de Taylor y minimiza esta aproximación en cada paso, lo que requiere resolver el sistema lineal definido por la Hessiana:

$$x_{k+1} = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

Este método tiene convergencia cuadrática (muy rápida) cerca del óptimo, pero es costoso debido al cálculo e inversión de la matriz Hessiana.

```
# Implementación del Método de Newton en Python
def metodo_newton(f, grad_f, hess_f, x_init, max_iter=50, tol=1e-6):
    x = np.array(x_init, dtype=float)
    historial = [x.copy()]

    for i in range(max_iter):
        grad = grad_f(x)
        if np.linalg.norm(grad) < tol:
            print(f"Newton: Convergencia alcanzada en la iteración {i}")
            break
        hess = hess_f(x)
        # Resolver el sistema lineal H * p = -grad para evitar inversión explícita
        p = np.linalg.solve(hess, -grad)
        x = x + p
```

```
historial.append(x.copy())

return x, np.array(historial)
```

2.4 Optimización Científica con SciPy

En la práctica, se utilizan solvers optimizados. En Python, la librería `scipy.optimize` proporciona la función `minimize` que implementa algoritmos avanzados como BFGS (cuasi-Newton) o Nelder-Mead.

```
from scipy.optimize import minimize

# Definición de la función de Rosenbrock (clásica para test de optimización)
def rosenbrock(x):
    return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2

# Optimización usando BFGS
x_init = [1.3, 0.7]
res = minimize(rosenbrock, x_init, method='BFGS')

print("Resultado de la optimización:")
print(res)
```

3 Análisis Convexo

El **análisis convexo** es el estudio de conjuntos y funciones convexas. En optimización, la convexidad es una propiedad extraordinaria: garantiza que cualquier mínimo local es, obligatoriamente, un mínimo global. Esto hace que los problemas de optimización convexa sean computacionalmente resolubles de forma eficiente y fiable.

3.1 Conjuntos Convexos

i Definición de Conjunto Convexo

Un conjunto $C \subseteq \mathbb{R}^n$ es **convexo** si el segmento de recta que une cualquier par de puntos en C se encuentra totalmente contenido en C . Matemáticamente:

$$\forall x, y \in C, \forall \theta \in [0, 1] \implies \theta x + (1 - \theta)y \in C$$

3.1.1 Ejemplos Clásicos de Conjuntos Convexos

1. **Espacios y Subespacios Afines:** Rectas, planos y sus generalizaciones.
2. **Hiperplanos:** El conjunto de puntos que satisfacen un producto escalar constante:

$$H = \{x \in \mathbb{R}^n \mid a^T x = b\} \quad (a \in \mathbb{R}^n, a \neq 0, b \in \mathbb{R})$$

3. **Semiespacios:** La región delimitada por un hiperplano:

$$S_1 = \{x \in \mathbb{R}^n \mid a^T x \leq b\} \quad \text{o} \quad S_2 = \{x \in \mathbb{R}^n \mid a^T x \geq b\}$$

4. **Poliedros:** La intersección de un número finito de semiespacios e hiperplanos (el conjunto factible de problemas de programación lineal):

$$P = \{x \in \mathbb{R}^n \mid Ax \leq b, Cx = d\}$$

```
# Verificación e ilustración gráfica de combinaciones convexas en Python
import numpy as np
import matplotlib.pyplot as plt

# Puntos extremos
x = np.array([1, 2])
y = np.array([4, 6])

# Combinación convexa con diferentes valores de theta
thetas = np.linspace(0, 1, 10)
segmento = np.array([theta * x + (1 - theta) * y for theta in thetas])

plt.figure(figsize=(6, 4))
plt.plot(segmento[:, 0], segmento[:, 1], 'o--', color='darkgreen', label='Segmento Convexo')
plt.scatter([x[0], y[0]], [x[1], y[1]], color='red', s=80, zorder=5, label='Puntos Extremos')
plt.title('Segmento Convexo entre dos Puntos en  $\mathbb{R}^2$ ')
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.legend()
plt.grid(True)
plt.show()
```

3.2 Funciones Convexas

i Definición de Función Convexa

Una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$ es **convexa** si su dominio $\text{dom } f$ es un conjunto convexo y para todos los puntos en su dominio se cumple:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y) \quad \forall \theta \in [0, 1]$$

Si la desigualdad es estricta para $x \neq y$ y $\theta \in (0, 1)$, se dice que es **estrictamente convexa**.

3.2.1 Caracterización mediante Derivadas

Para funciones diferenciables, la convexidad se puede caracterizar localmente:

- **Condición de Primer Orden:** Una función diferenciable f es convexa si y solo si:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) \quad \forall x, y \in \text{dom } f$$

Interpretación geométrica: La aproximación por plano tangente (afín) siempre se encuentra por debajo de la función.

- **Condición de Segundo Orden:** Una función dos veces diferenciable f es convexa si y solo si su dominio es convexo y su Hessiana es semidefinida positiva en todo punto:

$$\nabla^2 f(x) \succeq 0 \quad \forall x \in \text{dom } f$$

3.3 Operaciones que Preservan la Convexidad

Para construir nuevas funciones y conjuntos convexos a partir de otros conocidos:

3.3.1 En Conjuntos:

- **Intersección:** La intersección de cualquier familia de conjuntos convexos es convexa.
- **Funciones Afines:** Si C es convexo, su imagen bajo una transformación afín $f(x) = Ax + b$ es convexa.

3.3.2 En Funciones:

- **Suma ponderada no negativa:** Si f_1, f_2 son convexas y $w_1, w_2 \geq 0$, entonces $w_1 f_1 + w_2 f_2$ es convexa.
- **Supremo puntual (Máximo):** Si $f_1, \dots, f_k$ son convexas, la función máximo:

$$f(x) = \max\{f_1(x), f_2(x), \dots, f_k(x)\}$$

es convexa.

- **Composición con funciones crecientes:** Si $g : \mathbb{R}^n \rightarrow \mathbb{R}$ es convexa y $h : \mathbb{R} \rightarrow \mathbb{R}$ es convexa y creciente, entonces $f(x) = h(g(x))$ es convexa.

4 Optimización con Restricciones y Condiciones KKT

La mayoría de problemas prácticos implican optimizar bajo recursos o límites limitados. La **optimización restringida** introduce desigualdades e igualdades matemáticas que el vector de variables de decisión debe cumplir.

4.1 Formulación del Problema

Consideramos el problema estándar de minimización con restricciones de desigualdad e igualdad:

$$\begin{array}{ll} \underset{x \in \mathbb{R}^n}{\text{minimizar}} & f(x) \\ \text{sujeto a} & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{array}$$

4.1.1 Restricciones Activas e Inactivas

i Definición de Restricción Activa

Una restricción de desigualdad $g_i(x) \leq 0$ se dice que está **activa** en un punto factible x si se cumple con igualdad estricta:

$$g_i(x) = 0$$

Si $g_i(x) < 0$, la restricción está **inactiva** en ese punto.

Las restricciones de igualdad $h_j(x) = 0$ siempre están activas en cualquier punto factible.

4.2 La Función Lagrangiana y Multiplicadores de Lagrange

La **función Lagrangiana** combina la función objetivo y las restricciones mediante pesos llamados **multiplicadores de Lagrange** (o variables duales):

$$L(x, \lambda, \nu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{j=1}^p \nu_j h_j(x)$$

Donde: - $\lambda_i \geq 0$ son los multiplicadores asociados a las restricciones de desigualdad $g_i(x) \leq 0$.
- $\nu_j \in \mathbb{R}$ son los multiplicadores asociados a las restricciones de igualdad $h_j(x) = 0$.

4.3 Condiciones de Karush-Kuhn-Tucker (KKT)

Las condiciones KKT son fundamentales y proporcionan las condiciones necesarias de primer orden para que un punto x^* sea óptimo.

4.3.1 Teorema de KKT (Condiciones Necesarias)

Si x^* es un óptimo local y el problema cumple ciertas condiciones de regularidad (llamadas *cualificaciones de restricciones*, como la independencia lineal de los gradientes de las restricciones activas - LICQ), entonces existen vectores de multiplicadores λ^* y ν^* tales que se cumplen las siguientes cuatro condiciones KKT:

1. **Estacionariedad (Gradiente Nulo de la Lagrangiana):**

$$\nabla_x L(x^*, \lambda^*, \nu^*) = \nabla f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(x^*) + \sum_{j=1}^p \nu_j^* \nabla h_j(x^*) = 0$$

2. **Factibilidad Primal:**

$$g_i(x^*) \leq 0, \quad i = 1, \dots, m \quad \text{y} \quad h_j(x^*) = 0, \quad j = 1, \dots, p$$

3. **Factibilidad Dual (Multiplicadores no negativos para desigualdades):**

$$\lambda_i^* \geq 0, \quad i = 1, \dots, m$$

4. **Holgura Complementaria:**

$$\lambda_i^* g_i(x^*) = 0, \quad i = 1, \dots, m$$

Implicación: Si una restricción de desigualdad no está activa en el óptimo ($g_i(x^*) < 0$), su multiplicador asociado debe ser cero ($\lambda_i^* = 0$).

4.3.2 Condiciones Suficientes de KKT bajo Convexidad

Si el problema original es **convexo** (es decir, $f(x)$ y $g_i(x)$ son convexas y $h_j(x)$ son afines), entonces las condiciones KKT no solo son necesarias, sino que son **suficientes** para garantizar que el punto x^* es un **mínimo global**.

4.4 Dualidad Lagrangiana

4.4.1 La Función Dual de Lagrange

La función dual de Lagrange $q(\lambda, \nu)$ se define como el ínfimo de la Lagrangiana sobre x :

$$q(\lambda, \nu) = \inf_{x \in \mathbb{R}^n} L(x, \lambda, \nu)$$

La función dual tiene una propiedad importante: es siempre cóncava, incluso si el problema original no es convexo. Además, para cualquier par factible $(\lambda \geq 0, \nu)$, la función dual acota inferiormente al valor óptimo primal p^* :

$$q(\lambda, \nu) \leq p^*$$

4.4.2 El Problema Dual de Lagrange

El problema dual consiste en encontrar la mejor cota inferior:

$$\begin{array}{ll} \underset{\lambda, \nu}{\text{maximizar}} & q(\lambda, \nu) \\ \text{sujeto a} & \lambda_i \geq 0, \quad i = 1, \dots, m \end{array}$$

- **Dualidad Débil:** Siempre se cumple que el valor óptimo dual d^* es menor o igual al óptimo primal p^* : $d^* \leq p^*$.
- **Dualidad Fuerte:** Si $d^* = p^*$, se dice que se cumple la dualidad fuerte. En problemas convexos, bajo la **condición de Slater** (existencia de un punto estrictamente factible tal que $g_i(x) < 0$ para restricciones no afines), se garantiza la dualidad fuerte.

5 Programación Lineal, Cuadrática y CVXPY

En este capítulo estudiaremos cómo aplicar las condiciones KKT a las familias de problemas de optimización convexa más comunes e importantes en la práctica: la **Programación Lineal (LP)** y la **Programación Cuadrática (QP)**. Posteriormente, aprenderemos a formular y resolver estos modelos en Python mediante la librería **CVXPY**.

5.1 Programación Lineal (LP)

La Programación Lineal trata de minimizar (o maximizar) una función lineal sujeta a restricciones de igualdad y desigualdad lineales:

$$\begin{array}{ll} \underset{x \in \mathbb{R}^n}{\text{minimizar}} & c^T x \\ \text{sujeto a} & Ax \leq b \\ & Cx = d \end{array}$$

Donde $c \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $C \in \mathbb{R}^{p \times n}$ y $d \in \mathbb{R}^p$.

5.1.1 Condiciones KKT para LP

La función Lagrangiana de este problema es:

$$L(x, \lambda, \nu) = c^T x + \lambda^T (Ax - b) + \nu^T (Cx - d)$$

Aplicando las condiciones KKT: 1. **Estacionariedad:** $\nabla_x L = c + A^T \lambda + C^T \nu = 0$ 2. **Factibilidad primal:** $Ax \leq b$ y $Cx = d$ 3. **Factibilidad dual:** $\lambda \geq 0$ 4. **Holgura complementaria:** $\lambda_i (a_i^T x - b_i) = 0 \quad \forall i = 1, \dots, m$

5.2 Programación Cuadrática (QP)

La Programación Cuadrática implica minimizar una función objetivo cuadrática sujeta a restricciones lineales de igualdad y desigualdad:

$$\begin{array}{ll} \underset{x \in \mathbb{R}^n}{\text{minimizar}} & \frac{1}{2}x^T Px + q^T x \\ \text{sujeto a} & Ax \leq b \\ & Cx = d \end{array}$$

Donde $P \in \mathbb{R}^{n \times n}$ es una matriz simétrica. Para que el problema sea convexo y tenga un mínimo global garantizado, la matriz P debe ser **semidefinida positiva** ($P \succeq 0$).

5.2.1 Condiciones KKT para QP

La Lagrangiana es:

$$L(x, \lambda, \nu) = \frac{1}{2}x^T Px + q^T x + \lambda^T (Ax - b) + \nu^T (Cx - d)$$

Condiciones KKT: 1. **Estacionariedad:** $Px + q + A^T \lambda + C^T \nu = 0$ 2. **Factibilidad primal:** $Ax \leq b$ y $Cx = d$ 3. **Factibilidad dual:** $\lambda \geq 0$ 4. **Holgura complementaria:** $\lambda_i (a_i^T x - b_i) = 0 \quad \forall i = 1, \dots, m$

5.3 Resolución de Problemas en Python con CVXPY

CVXPY es un lenguaje de modelado en Python para optimización convexa que permite escribir problemas de manera natural, similar a la notación matemática.

5.3.1 Ejemplo 1: Resolución de una Programación Lineal (LP)

Resolveremos el siguiente problema:

$$\begin{array}{ll} \underset{x_1, x_2}{\text{maximizar}} & 3x_1 + 4x_2 \\ \text{sujeto a} & x_1 + 2x_2 \leq 4 \\ & 3x_1 - x_2 \geq 1 \\ & x_1 \geq 0, x_2 \geq 0 \end{array}$$

```

import cvxpy as cp

# 1. Definir variables de decisión
x = cp.Variable(2, nonneg=True) # Equivalente a x1 >= 0, x2 >= 0

# 2. Definir restricciones
restricciones = [
    x[0] + 2 * x[1] <= 4,
    3 * x[0] - x[1] >= 1
]

# 3. Definir función objetivo
objetivo = cp.Maximize(3 * x[0] + 4 * x[1])

# 4. Definir y resolver el problema
problema = cp.Problem(objetivo, restricciones)
problema.solve()

# 5. Mostrar resultados
print(f"Estado del problema: {problema.status}")
print(f"Valor óptimo de la función objetivo: {problema.value:.4f}")
print(f"Valores óptimos de las variables: x1 = {x.value[0]:.4f}, x2 = {x.value[1]:.4f}")
print(f"Valores óptimos de las variables duales (multiplicadores): {restricciones[0].dual_val}")

```

5.3.2 Ejemplo 2: Resolución de una Programación Cuadrática (QP)

Resolveremos la minimización de la distancia al origen sujeta a restricciones lineales (un problema QP convexo):

$$\begin{array}{ll}
 \underset{x_1, x_2}{\text{minimizar}} & \frac{1}{2}(x_1^2 + x_2^2) \\
 \text{sujeto a} & x_1 + x_2 \geq 1
 \end{array}$$

```

import cvxpy as cp
import numpy as np

x = cp.Variable(2)

# P = identidad, q = cero
P = np.eye(2)
q = np.zeros(2)

```

```

# Función objetivo:  $1/2 * x^T * P * x$ 
objetivo = cp.Minimize(0.5 * cp.quad_form(x, P))

# Restricciones
restricciones = [
    x[0] + x[1] >= 1
]

problema = cp.Problem(objetivo, restricciones)
problema.solve()

print(f"Estado: {problema.status}")
print(f"Valor óptimo: {problema.value:.4f}")
print(f"x óptimo: {x.value}")

```

6 Optimización en Ciencia de Datos: Regresión y SVM

La optimización es el motor matemático del aprendizaje automático. En este capítulo estudiaremos cómo formular y resolver dos problemas fundamentales de la ciencia de datos desde la perspectiva de la optimización convexa: la **Regresión Lineal** y las **Máquinas de Vectores de Soporte (SVM)**.

6.1 Regresión Lineal y Regularización

Dada una matriz de características $X \in \mathbb{R}^{m \times n}$ y un vector de etiquetas $y \in \mathbb{R}^m$, el problema de mínimos cuadrados ordinarios busca minimizar la norma del residuo:

$$\underset{w \in \mathbb{R}^n, b \in \mathbb{R}}{\text{minimizar}} \quad \|Xw + b\mathbf{1} - y\|_2^2$$

Para evitar el sobreajuste y estabilizar el sistema cuando hay multicolinealidad, se añaden términos de penalización (regularización):

1. **Ridge Regression (L2)**: Minimiza $\|Xw + b\mathbf{1} - y\|_2^2 + \gamma\|w\|_2^2$ (convexo, diferenciable).
 2. **Lasso Regression (L1)**: Minimiza $\|Xw + b\mathbf{1} - y\|_2^2 + \gamma\|w\|_1$ (convexo, no diferenciable en cero, genera soluciones dispersas/sparse).
-

6.2 Máquinas de Vectores de Soporte (SVM)

El objetivo de una SVM es encontrar un hiperplano de separación en $\mathbb{R}^n$ para clasificar datos binarios $y_i \in \{-1, +1\}$ maximizando el **margen** (la distancia al punto más cercano de cada clase).

6.2.1 1. Caso Linealmente Separable (Margen Máximo)

El margen es inversamente proporcional a la norma del vector de pesos: $M = 2/\|w\|_2$. Maximizar el margen equivale a minimizar la norma al cuadrado, lo que da lugar al problema QP convexo restringido:

$$\begin{aligned} & \underset{w \in \mathbb{R}^n, b \in \mathbb{R}}{\text{minimizar}} && \frac{1}{2} \|w\|_2^2 \\ & \text{sujeto a} && y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, m \end{aligned}$$

```
# Formulación de SVM separable en CVXPY
import cvxpy as cp
import numpy as np

# Datos de prueba simulados
X_data = np.array([[1, 2], [2, 3], [3, 1], [6, 8], [7, 7], [8, 9]])
y_data = np.array([-1, -1, -1, 1, 1, 1])

m, n = X_data.shape
w = cp.Variable(n)
b = cp.Variable()

restricciones = [
    y_data[i] * (X_data[i] @ w + b) >= 1 for i in range(m)
]

objetivo = cp.Minimize(0.5 * cp.sum_squares(w))
prob = cp.Problem(objetivo, restricciones)
prob.solve()

print(f"w óptimo: {w.value}")
print(f"b óptimo: {b.value}")
```

6.2.2 2. Caso No Separable (Margen Blando)

Si los datos no son perfectamente separables, introducimos variables de holgura $\xi_i \geq 0$ que penalizan los puntos mal clasificados. El parámetro $C > 0$ controla el equilibrio entre la maximización del margen y la tolerancia al error:

$$\begin{array}{ll}
\underset{w, b, \xi}{\text{minimizar}} & \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^m \xi_i \\
\text{sujeto a} & y_i(w^T x_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, m \\
& \xi_i \geq 0, \quad i = 1, \dots, m
\end{array}$$

6.2.3 3. Formulación Dual de SVM

Mediante la dualidad lagrangiana, el problema de SVM se puede reescribir de forma dual, donde las variables de optimización son los multiplicadores α_i :

$$\begin{array}{ll}
\underset{\alpha \in \mathbb{R}^m}{\text{maximizar}} & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j x_i^T x_j \\
\text{sujeto a} & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, m \\
& \sum_{i=1}^m \alpha_i y_i = 0
\end{array}$$

Esta formulación dual es de gran valor por dos motivos: - **Vectores de soporte:** Los puntos correspondientes a $\alpha_i > 0$ son los únicos que determinan el hiperplano (los *vectores de soporte*). - **El Truco del Kernel:** Podemos sustituir el producto escalar $x_i^T x_j$ por una función Kernel $K(x_i, x_j)$ para mapear los datos de entrada a un espacio de mayor dimensión donde sean linealmente separables (por ejemplo, usando kernels polinomiales o Gaussianos RBF).

```
# Ejemplo de uso con scikit-learn para comparar
from sklearn.svm import SVC

clf = SVC(kernel='linear', C=1.0)
clf.fit(X_data, y_data)

print(f"coef_ (w) de sklearn: {clf.coef_}")
print(f"intercept_ (b) de sklearn: {clf.intercept_}")
```

7 Introducción al Análisis de Redes y NetworkX

El **análisis de redes** es una rama de las matemáticas discretas y la optimización que estudia sistemas de objetos interconectados. Muchas redes físicas, biológicas y sociales (como la red eléctrica, el tráfico vial, internet o las relaciones de amistad) se modelan matemáticamente mediante **grafos**.

7.1 Conceptos Básicos de Teoría de Grafos

Un **grafo** G es un par ordenado $G = (V, E)$ donde: - V es el conjunto de **vértices** o **nodos** ($V = \{v_1, v_2, \dots, v_n\}$). - E es el conjunto de **aristas** o **arcos** ($E \subseteq V \times V$), que representan las conexiones entre nodos.

7.1.1 Tipos de Grafos

1. **Grafo No Dirigido:** Las conexiones no tienen dirección; si $(u, v) \in E$, entonces también $(v, u) \in E$. Se representan gráficamente como segmentos.
 2. **Grafo Dirigido (Digrafo):** Las conexiones tienen un sentido; el arco (u, v) va del nodo origen u al nodo destino v . Se representan como flechas.
 3. **Grafo Valorado (o Red):** A cada arista o arco se le asigna un número real llamado **peso** o **coste** $w(u, v)$ (por ejemplo, la distancia en kilómetros o el retardo de un canal de datos).
-

7.2 Representación de Grafos en Ordenador

Para procesar grafos mediante algoritmos, existen dos representaciones clásicas:

1. **Matriz de Adyacencia** ($A \in \mathbb{R}^{n \times n}$): Es una matriz donde la entrada a_{ij} es 1 (o el peso w) si existe conexión entre el nodo i y el j , y 0 en caso contrario. Es idónea para grafos densos.
 2. **Lista de Adyacencia**: Asocia a cada nodo una lista con sus vecinos. Es eficiente en memoria para grafos dispersos.
-

7.3 Introducción a NetworkX en Python

NetworkX es la librería estándar de Python para la creación, manipulación y estudio de redes complejas.

```
import networkx as nx
import matplotlib.pyplot as plt

# 1. Crear un grafo no dirigido
G = nx.Graph()

# 2. Añadir nodos
G.add_node("A")
G.add_nodes_from(["B", "C", "D"])

# 3. Añadir aristas con pesos
G.add_edge("A", "B", weight=4.5)
G.add_edges_from([
    ("B", "C", {"weight": 2.0}),
    ("C", "D", {"weight": 1.5}),
    ("D", "A", {"weight": 3.0})
])

# 4. Obtener información básica
print(f"Nodos: {G.nodes()}")
print(f"Aristas: {G.edges(data=True)}")
print(f"Número de nodos: {G.number_of_nodes()}")
print(f"Número de aristas: {G.number_of_edges()}")

# 5. Dibujar el grafo
pos = nx.spring_layout(G) # Distribución de fuerza elástica para los nodos
nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=800, font_weight='bold')
labels = nx.get_edge_attributes(G, 'weight')
```

```
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
plt.title("Grafo Valorado en NetworkX")
plt.show()
```

8 Problemas de Camino Mínimo

El problema del **camino mínimo** consiste en encontrar la ruta de menor coste o distancia acumulada entre un nodo origen s y un nodo destino t (o entre todos los pares de nodos) en un grafo valorado dirigiendo o no.

8.1 Algoritmos Fundamentales

8.1.1 1. Algoritmo de Dijkstra

Diseñado para grafos con pesos **no negativos** ($w(u, v) \geq 0$). Es un algoritmo codicioso (greedy) que mantiene una estimación de la distancia mínima a cada nodo y explora los vecinos del nodo más cercano aún no procesado. - **Complejidad:** $O(|V|^2)$ o $O(|E| + |V| \log |V|)$ utilizando colas de prioridad (heaps).

8.1.2 2. Algoritmo de Bellman-Ford

Acepta pesos **negativos** en las aristas. Además, es capaz de detectar la existencia de **ciclos de coste negativo** (circuitos donde dar una vuelta reduce el coste acumulado, lo que hace que el camino mínimo tienda a $-\infty$ y no esté bien definido). - **Complejidad:** $O(|V||E|)$.

8.1.3 3. Algoritmo de Floyd-Warshall

Es un algoritmo de programación dinámica que calcula los caminos mínimos entre **todos los pares de nodos** en un solo paso. Admite pesos negativos pero no ciclos negativos. - **Complejidad:** $O(|V|^3)$.

8.2 Resolución de Caminos Mínimos con NetworkX

NetworkX implementa todos estos algoritmos en funciones optimizadas de alta jerarquía.

```
import networkx as nx
import matplotlib.pyplot as plt

# Crear una red con pesos de ejemplo
G = nx.DiGraph()
G.add_edge("s", "A", weight=3)
G.add_edge("s", "B", weight=5)
G.add_edge("A", "B", weight=1)
G.add_edge("A", "C", weight=6)
# Arista con peso negativo para ilustrar
G.add_edge("B", "C", weight=-2)
G.add_edge("C", "t", weight=4)

# 1. Calcular camino mínimo con Dijkstra (no recomendado si hay pesos negativos)
try:
    path_dij = nx.dijkstra_path(G, "s", "t")
    dist_dij = nx.dijkstra_path_length(G, "s", "t")
    print(f"Dijkstra: Ruta = {path_dij}, Distancia = {dist_dij}")
except ValueError as e:
    print(f"Error en Dijkstra: {e} (debido al peso negativo)")

# 2. Calcular camino mínimo con Bellman-Ford (robusto frente a pesos negativos)
path_bf = nx.bellman_ford_path(G, "s", "t")
dist_bf = nx.bellman_ford_path_length(G, "s", "t")
print(f"Bellman-Ford: Ruta = {path_bf}, Distancia = {dist_bf}")

# 3. Detectar ciclos negativos
tiene_ciclo_negativo = nx.negative_edge_cycle(G)
print(f"¿Tiene ciclos negativos?: {tiene_ciclo_negativo}")

# 4. Dibujar y destacar el camino mínimo
pos = {"s": (0, 1), "A": (1, 2), "B": (1, 0), "C": (2, 1), "t": (3, 1)}
nx.draw(G, pos, with_labels=True, node_color='lightgrey', node_size=700)

# Destacar nodos y aristas del camino mínimo (s -> A -> B -> C -> t)
edges_camino = [("s", "A"), ("A", "B"), ("B", "C"), ("C", "t")]
nx.draw_networkx_nodes(G, pos, nodelist=path_bf, node_color='orange', node_size=700)
nx.draw_networkx_edges(G, pos, edgelist=edges_camino, edge_color='orange', width=3)
```

```
labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
plt.title("Ruta del Camino Mínimo Destacada")
plt.show()
```


9 Problemas de Flujo en Redes

Los problemas de **flujo en redes** modelan el transporte de un recurso (fluido, datos, productos, energía) a través de una red de tuberías o canales con capacidades limitadas.

9.1 El Problema de Flujo Máximo

Consiste en determinar la cantidad máxima de recurso que se puede enviar desde un nodo origen (fuente s) hasta un nodo destino (sumidero t), respetando las capacidades de cada canal y la conservación de flujo en los nodos intermedios.

9.1.1 Algoritmo de Ford-Fulkerson

El método se basa en encontrar **caminos aumentadores** (rutas desde s a t donde todavía queda capacidad residual) e ir empujando flujo a través de ellas hasta que no queden más caminos disponibles. - Cuando se implementa buscando el camino más corto mediante BFS, el algoritmo se conoce como **Edmonds-Karp** y tiene una complejidad garantizada de $O(|V||E|^2)$.

9.1.2 Teorema de Flujo Máximo - Corte Mínimo

! Teorema Max-Flow Min-Cut

El flujo máximo que puede circular desde s a t en una red es exactamente igual a la capacidad del **corte mínimo** que separa a s de t . Un corte mínimo es una partición de los nodos en dos conjuntos S (que contiene a s) y T (que contiene a t) tal que la suma de las capacidades de los arcos que van de S a T es mínima.

9.2 Problema de Flujo a Coste Mínimo (Min-Cost Flow)

En este problema, cada arco de la red tiene tanto una **capacidad** (límite superior de flujo) como un **coste unitario** por enviar flujo a través de él. El objetivo es satisfacer una demanda de recurso determinada en los nodos de destino a partir de los nodos de origen minimizando el coste total acumulado.

9.3 Modelado y Resolución con NetworkX

```
import networkx as nx

# 1. Crear la red de flujo
G = nx.DiGraph()
G.add_edge("s", "A", capacity=10)
G.add_edge("s", "B", capacity=10)
G.add_edge("A", "B", capacity=2)
G.add_edge("A", "t", capacity=8)
# Arco con gran capacidad
G.add_edge("B", "t", capacity=12)

# 2. Resolver Flujo Máximo
flow_value, flow_dict = nx.maximum_flow(G, "s", "t")
print(f"Flujo Máximo: {flow_value}")
print("Flujo por arco:")
for u, neighbors in flow_dict.items():
    for v, flow in neighbors.items():
        if flow > 0:
            print(f" {u} -> {v}: {flow}/{G[u][v]['capacity']}")

# 3. Encontrar el Corte Mínimo
cut_value, partitions = nx.minimum_cut(G, "s", "t")
print(f"\nValor del Corte Mínimo: {cut_value}")
print(f"Partición de nodos S: {partitions[0]}")
print(f"Partición de nodos T: {partitions[1]}")
```

10 Problemas de Emparejamiento (Matching)

Un **emparejamiento** (o matching) en un grafo $G = (V, E)$ es un conjunto de aristas $M \subseteq E$ independientes, es decir, aristas que no comparten ningún nodo en común.

10.1 Tipos de Emparejamientos

1. **Emparejamiento Máximo:** Un emparejamiento M tal que contiene el mayor número posible de aristas.
 2. **Emparejamiento Perfecto:** Un emparejamiento tal que todos los nodos del grafo están conectados a alguna arista de M (es decir, $|M| = |V|/2$).
-

10.2 Emparejamiento en Grafos Bipartitos

Un grafo es **bipartito** si su conjunto de nodos V se puede dividir en dos conjuntos disjuntos U y W ($V = U \cup W$) tal que no existen aristas que unan nodos dentro del mismo conjunto.

El problema del **emparejamiento máximo bipartito** consiste en emparejar el mayor número de elementos de U con elementos de W (por ejemplo, asignar trabajadores a tareas de forma óptima). - Este problema se puede resolver de manera muy eficiente reformulándolo como un **problema de flujo máximo** añadiendo una fuente artificial conectada a todos los nodos de U y un sumidero artificial conectado a todos los nodos de W .

10.3 Resolución de Emparejamientos con NetworkX

NetworkX proporciona algoritmos eficientes especializados para grafos bipartitos.

```
import networkx as nx
from networkx.algorithms import bipartite

# 1. Crear un grafo bipartito
B = nx.Graph()
# Nodos de la izquierda (Trabajadores)
trabajadores = ["T1", "T2", "T3", "T4"]
# Nodos de la derecha (Tareas)
tareas = ["Tarea_A", "Tarea_B", "Tarea_C", "Tarea_D"]

B.add_nodes_from(trabajadores, bipartite=0)
B.add_nodes_from(tareas, bipartite=1)

# Aristas: compatibilidades o habilidades
B.add_edges_from([
    ("T1", "Tarea_A"), ("T1", "Tarea_B"),
    ("T2", "Tarea_B"), ("T2", "Tarea_C"),
    ("T3", "Tarea_C"),
    ("T4", "Tarea_A"), ("T4", "Tarea_D")
])

# 2. Calcular emparejamiento máximo
matching = bipartite.maximum_matching(B, top_nodes=trabajadores)
print("Emparejamiento Máximo calculado:")
for k, v in matching.items():
    # El diccionario devuelto es bidireccional, filtramos para mostrar solo una dirección
    if k in trabajadores:
        print(f" {k} asignado a: {v}")
```

11 Problemas de Recorridos (Tours)

Los problemas de **recorridos o rutas** estudian cómo viajar a través de los nodos o aristas de un grafo cumpliendo ciertas restricciones de visita y minimizando la distancia o coste total.

11.1 Recorridos Eulerianos y el Cartero Chino

i Definición de Ciclo Euleriano

Un **ciclo euleriano** es un camino cerrado que recorre **cada arista** del grafo exactamente una vez.

- **Condición de Euler:** Un grafo conexo no dirigido admite un ciclo euleriano si y solo si **todos sus nodos tienen grado par**. Si tiene exactamente dos nodos de grado impar, admite un *camino euleriano* (con origen y destino en dichos nodos impares).
 - **Problema del Cartero Chino (CPP):** Si el grafo tiene nodos de grado impar, no es euleriano. El CPP consiste en encontrar la ruta cerrada más corta que pase por cada arista al menos una vez (lo que requiere duplicar algunas aristas de coste mínimo).
-

11.2 Recorridos Hamiltonianos y el Viajante de Comercio (TSP)

i Definición de Ciclo Hamiltoniano

Un **ciclo hamiltoniano** es un camino cerrado que visita **cada nodo** del grafo exactamente una vez.

- **Problema del Viajante de Comercio (TSP):** Consiste en encontrar el ciclo hamiltoniano de coste mínimo en un grafo completo valorado. A diferencia de los ciclos eulerianos (fáciles de calcular), el TSP es un problema **NP-difícil**, lo que significa que no se conoce ningún algoritmo exacto en tiempo polinomial.
- **Estrategias de resolución:** Heurísticas de vecino más cercano, búsqueda local (2-opt), algoritmos de aproximación (como el algoritmo de Christofides, que garantiza un coste no superior a 1.5 veces el óptimo en espacios métricos) y solvers exactos de programación lineal entera.

11.3 Implementación en Python con NetworkX

```
import networkx as nx

# 1. Crear un grafo euleriano (todos los nodos con grado par)
G = nx.cycle_graph(4) # Ciclo simple de 4 nodos (grado 2 en todos)
es_euleriano = nx.is_eulerian(G)
print(f"¿El grafo G es euleriano?: {es_euleriano}")

if es_euleriano:
    recorrido = list(nx.eulerian_circuit(G))
    print(f"Recorrido Euleriano: {recorrido}")

# 2. Resolución del Viajante de Comercio (TSP) aproximado
# Crear un grafo completo valorado
K = nx.complete_graph(4)
# Añadir pesos
pesos = {
    (0, 1): 10, (0, 2): 15, (0, 3): 20,
    (1, 2): 35, (1, 3): 25, (2, 3): 30
}
for (u, v), w in pesos.items():
    K[u][v]['weight'] = w

# Resolver usando aproximación de Christofides (requiere pesos métricos)
from networkx.algorithms.approximation import traveling_salesperson_problem
ruta_tsp = traveling_salesperson_problem(K, weight='weight', cycle=True)
print(f"\nRuta aproximada del TSP: {ruta_tsp}")
```